

Skeleton Proposal for Thread-Local Storage (TLS)

ISO/IEC JTC1 SC22 WG21 P0108R0 - 2015-09-24

Paul E. McKenney, paulmck@linux.vnet.ibm.com

JF Bastien, jfb@google.com

TBD

Introduction

This document is a follow-on to [N4376](#), and provides an initial description of a potential solution to the TLS problem statement implied by that document.

Summary of Problem Statement

We expect that lightweight executors will have problems with TLS as currently envisioned and implemented. For example, some types of executors nest hierarchically, so that a number of light-weight executors might run in the context of a single heavy-weight `std::thread`. If a given function accesses TLS, and is called both from the context of a `std::thread` and from the context of a task executing within an `std::thread`, what should its TLS accesses do? If the instances invoked from a task access task-level TLS data, the function must do different things when invoked in different contexts. If the `std::thread`-level TLS data is accessed, then the task-level accesses might introduce data races and thus undefined behavior.

This also can interact with signal handling. To see this, suppose that a signal arrives at a `std::thread` while that `std::thread` is running a light-weight executor, for example, a task. The signal handler will likely conceptually be part of the `std::thread` rather than the task. This would imply some additional context switching at signal-handler start and end.

TLS is most especially a problem for light-weight executors implementing same-instruction-multiple-data (SIMD) units and general-purpose graphical processing units (GPGPUs) because large programs can have very large amounts of TLS data, each item of which might have C++ constructors and destructors. Spending many milliseconds to run constructors and destructors for a SIMD computation that only takes a few microseconds to run is clearly not a reasonable way to achieve high performance.

GPGPU code often has longer runtimes, but they also tend to run extremely large numbers of threads, adding a memory-footprint problem to the constructor-destructor overhead problem. To make matters worse, in some environments, the constructors and destructors must be run on heavyweight CPUs rather than on the lightweight GPGPU hardware threads, which severely restricts the computational resources that can be applied to run constructors and destructors for GPGPU TLS data.

At the source-code level, it isn't generally knowable which executor a function is called from, or even if a function is called from multiple executors. It is left up to the programmer to write code which correctly accesses state for the executor(s) that the code will execute in. (In theory, we could of course use a TLS variable to record what type of executor was currently executing, but in practice that of course requires a TLS implementation that is efficient enough to be used by light-weight executors, and if we had that, we wouldn't be writing this paper.)

Tentative Goals

There are a number of possible ways of resolving this issue, as discussed in [N4376](#), however, this paper focuses on the possibility that TLS is an optional component of an executor. With this approach, `std::thread` implements TLS, but lighter-weight executors might choose not to.

For this approach, we put forward the following tentative goals:

1. Make TLS availability optional for light-weight executors, as noted above.
 1. Modify the standard library so as to minimize the number of standard library functions that are prohibited from within TLS-free executors.
 2. Maintain the performance and scalability of high-quality standard-library implementations.
2. Avoid source-code changes for existing code running in existing executors (such as `std::thread`) that provide TLS.
3. Avoid the need to recompile existing code running in existing executors (such as `std::thread`) that provide TLS.
4. Avoid API changes in the standard library. (C++ only, as it seems quite unlikely that this goal can be achieved in C.)
5. Recruit sanitizer developers to help identify issues in new code and in standard-library code related to this change.

The next section exercises these goals by attempting to apply them to the TLS `errno` facility as used by the standard math library, in the hope of sparking productive discussion. Note that when multiple lightweight executors run concurrently in the context of

a single `std::thread`, setting `errno` implicitly (and for some, surprisingly) invokes undefined behavior, so a fix is a matter of some importance. At a minimum, lightweight executors that do not support TLS need to state that attempts to access TLS results in undefined behavior.

The Curious Case of `errno` and the Standard Math Library

C++ provides a `per-std::thread` facility named `errno` (19.4) in order to provide POSIX compatibility. This is also required to allow C++'s standard math library (26) maintain compatibility with that of C. Section 7.12 of the C standard specifies that if `math_errhandling & MATH_ERRNO` is non-zero, indication of certain errors are available via `errno`. Furthermore, Section 19.4 of the C++ standard specifies that `errno` is provided on a per-thread basis. Therefore, `errno` is frequently implemented using TLS, which in turn means that the math library's use of `errno` forms an excellent initial test case for changes to TLS.

This section looks at the following approaches:

1. Restricting configuration.
2. Adding `errno` parameter via function overloading.
3. Adding `errno` to return value.

Restricting Configuration

One approach is to require that `math_errhandling & MATH_ERREXCEPT` be non-zero (as is required for IEC 60559) and that `math_errhandling & MATH_ERRNO` be zero in all cases where math library functions are invoked from executors that do not provide TLS. Note that `math_errhandling` is global and constant, which means that it cannot have different values in different contexts of the same execution. However, this approach cannot be used in conjunction with existing code that invokes math functions and tests `errno`. This could in turn be dealt with by forbidding use of code that checks for math errors using `errno`, but this would have the undesirable effect of acting as a barrier to the adoption of light-weight executors.

Adding `errno` Parameter Via Function Overloading

Another approach is to use function overloading, so that an additional `double sqrt(double, int *)` declaration could be used in light-weight executors. Note that in some implemnetations this could require modifying the underlying C library in order to bypass `errno` setting. Code invoked both from light-weight and heavy-weight executors would need to use the new delaration, but code invoked only from heavy-weight executors could continue using the old API, consistent with the goals preserving existing source and binary code. It is tempting to instead overload on the return value, but C++ of course does not support this notion. A (probably partial) list of new APIs is as follows:

- `double acos(double x, int *errnm);`
- `float acosf(float x, int *errnm);`
- `long double acosl(long double x, int *errnm);`
- `double asin(double x, int *errnm);`
- `float asinf(float x, int *errnm);`
- `long double asinl(long double x, int *errnm);`
- `double atan2(double y, double x, int *errnm);`
- `float atan2f(float y, float x, int *errnm);`
- `long double atan2l(long double y, long double x, int *errnm);`
- `double acosh(double xint *errnm);`
- `float acoshf(float xint *errnm);`
- `long double acoshl(long double xint *errnm);`
- `double atanh(double xint *errnm);`
- `float atanhf(float xint *errnm);`
- `long double atanh1(long double xint *errnm);`
- `double cosh(double xint *errnm);`
- `float coshf(float xint *errnm);`
- `long double coshl(long double xint *errnm);`
- `double sinh(double xint *errnm);`
- `float sinh1(long double xint *errnm);`
- `double exp(double xint *errnm);`
- `float expf(float xint *errnm);`
- `long double expl(long double xint *errnm);`
- `double exp2(double xint *errnm);`
- `float exp2f(float xint *errnm);`
- `long double exp2l(long double xint *errnm);`

- double expm1(double xint *errnm);
- float expm1f(float xint *errnm);
- long double expm1l(long double xint *errnm);
- int ilogb(double xint *errnm);
- int ilogbf(float xint *errnm);
- int ilogbl(long double xint *errnm);
- double log(double xint *errnm);
- float logf(float xint *errnm);
- long double logl(long double xint *errnm);
- double log10(double xint *errnm);
- float log10f(float xint *errnm);
- long double log10l(long double xint *errnm);
- double log1p(double xint *errnm);
- float log1pf(float xint *errnm);
- long double log1pl(long double xint *errnm);
- double log2(double xint *errnm);
- float log2f(float xint *errnm);
- long double log2l(long double xint *errnm);
- double logb(double xint *errnm);
- float logbf(float xint *errnm);
- long double logbl(long double xint *errnm);
- double scalbn(double x, int nint *errnm);
- float scalbnf(float x, int nint *errnm);
- long double scalbnl(long double x, int nint *errnm);
- double scalbln(double x, long int nint *errnm);
- float scalblnf(float x, long int nint *errnm);
- long double scalblnl(long double x, long int nint *errnm);
- double hypot(double x, double yint *errnm);
- float hypotf(float x, float yint *errnm);
- long double hypotl(long double x, long double yint *errnm);
- double pow(double x, double yint *errnm);
- float powf(float x, float yint *errnm);
- long double powl(long double x, long double yint *errnm);
- double sqrt(double xint *errnm);
- float sqrtf(float xint *errnm);
- long double sqrtl(long double xint *errnm);
- double erfc(double xint *errnm);
- float erfcf(float xint *errnm);
- long double erfcl(long double xint *errnm);
- double lgamma(double xint *errnm);
- float lgammaf(float xint *errnm);
- long double lgammal(long double xint *errnm);
- double tgamma(double xint *errnm);
- float tgammaf(float xint *errnm);
- long double tgammaal(long double xint *errnm);
- long int lrint(double xint *errnm);
- long int lrintf(float xint *errnm);
- long int lrintl(long double xint *errnm);
- long long int llrint(double xint *errnm);
- long long int llrintf(float xint *errnm);
- long long int llrintl(long double xint *errnm);
- long int lround(double xint *errnm);
- long int lroundf(float xint *errnm);
- long int lroundl(long double xint *errnm);
- long long int llround(double xint *errnm);
- long long int llroundf(float xint *errnm);
- long long int llroundl(long double xint *errnm);
- double fmod(double x, double yint *errnm);
- float fmodf(float x, float yint *errnm);
- long double fmodl(long double x, long double yint *errnm);
- double remainder(double x, double yint *errnm);
- float remainderf(float x, float yint *errnm);

- `long double remainderl(long double x, long double yint *errnm);`
- `double remquo(double x, double y, int *quooint *errnm);`
- `float remquof(float x, float y, int *quooint *errnm);`
- `long double remquol(long double x, long double y, int *quooint *errnm);`
- `double nextafter(double x, double yint *errnm);`
- `float nextafterf(float x, float yint *errnm);`
- `long double nextafterl(long double x, long double yint *errnm);`
- `double fdim(double x, double yint *errnm);`
- `float fdimf(float x, float yint *errnm);`
- `long double fdiml(long double x, long double yint *errnm);`
- `double fma(double x, double y, double zint *errnm);`
- `float fmaf(float x, float y, float zint *errnm);`
- `long double fmal(long double x, long double y, long double zint *errnm);`

Note that new APIs need be provided only for those math functions that set `errno`. Note also that because C does not provide function overloading, different names will need to be used should C adopt similar functionality.

One might expect some dissatisfaction with the invention of more than 100 new functions, especially given that a great many uses of these functions ignore `errno`. Although one can argue that ignoring `errno` is a bad idea, one might also expect strenuous objections to pointless modifications of existing `errno`-ignoring code.

Adding `errno` to Function Return Value

Another approach is to define an additional namespace containing definitions of these functions that return a tuple that includes both the normal return value and the `errno` value. For example:

```

1 std::tuple<T, errno_t> acos(T);
2
3 template<typename T> struct math_result {
4     explicit math_result(T);
5     explicit math_result(errno_t);
6     T operator T() const;
7     errno_t error() const;
8     // Implementation-defined.
9 };

```

This approach allows `errno`-ignoring code to run safely in light-weight executors, with modest changes for code that pays attention to `errno`. One way of preventing silent miscomputation by `errno`-ignoring code is to use exceptions, which this approach also supports. However, some might take exception to the use of exceptions, given that a number of current implementations of exceptions use, you guessed it, TLS!

Summary

This document has examined some ways to permit light-weight executors to avoid implementing TLS. Your ideas are more than welcome!

Future work includes handling of allocators (which introduces the problem of cross-executor freeing), `setjmp/longjmp`, locales, filesystems, signal handling, floating-point rounding modes (and everything else in `fenv`), and exceptions. The problem of nested executors that all provide TLS is also left unaddressed by this draft. In addition, and perhaps most important, future work includes guidelines and patterns to allow user code to work well with TLS in environments that include lightweight executors.

Acknowledgements

@@@

Additional Information

Floating-point state is stored on a per-thread basis, which means that if a light-weight executor can be preempted or migrated among `std::thread` instance, things like rounding modes and error/exception indications can be subject to unscheduled revision.